

JavaScript

INTRODUCTION TO JAVASCRIPT:

- JS is used to make webpages interactive & dynamic.
- It is a high-level, lightweight and scripting language.
- JS code runs inside the browser engine
- JS follows top-down approach
- It was introduced by Brendan Eich in 1995
- JS follows ECMAScript Standards
- Present version is ES2025

TYPES OF JS:

Inline JavaScript:

- Inline JavaScript is written **directly inside HTML elements**.
- It is usually used inside event attributes like onclick.

Internal JavaScript:

- Internal JavaScript is written **inside the HTML file**.
- It is placed within the <script> tag.
- We will write <script> tag just before closing </body> tag

External JavaScript:

- External JavaScript is written in a **separate .js file**.
- The file is linked to HTML using the <script src=""> tag.
- If we want to write <script src=""> tag in head then use **defer** keyword

INPUT & OUTPUT FUNCTIONS:

- Input functions: used to accept data from the user
 - confirm()
 - prompt()
- Output functions: used to display data to the user
 - alert()
 - document.write()
 - document.writeln()
 - console.log()

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Document</title>
  <script src="script.js" defer></script>
</head>
<body>
  <h2 onclick="console.log('hello')">We have started learning javascript</h2>
  <script>
    console.log("Hello Vemu")
    console.log('Vemu college')
    document.write("first line")
    document.writeln("second line")
    alert("web page loaded sucessfully")
    console.log(prompt("Enter number"))
    console.log(confirm("make sure"))
  </script>
</body>
</html>
```

JS COMMENTS:

- Single line Comments
- Multi Line Comments

CODE:

```
<script>
// This is a single-line comment

/*
This is a
multi-line comment
*/
</script>
```

EVENTS IN JS:

- onclick - execute when an element is clicked
- onload - runs automatically when the page (or element) finishes loading
- onchange - runs when the value of input element changes
- onmouseover - execute when mouse pointer enters an element
- onmouseout - execute when mouse pointer leaves an element
- onfocus - click inside input (input field gets focus)
- onblur - click outside input
- onsubmit - Executed when a form is submitted

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Document</title>
</head>
<body onload="alert('page loaded successfully')">
  <h1 onclick="console.log('onclick event')">onclick event</h1>
  <form action="" onsubmit="alert('form submitted')">
    <label for="">Name</label>
    <input type="text" onchange="alert('text changed')">
    <br><br>
    <label for="">Age</label>
    <input type="number" onmouseover="this.style.backgroundColor='pink'"
onmouseout="this.style.backgroundColor='yellow'"><br><br>
    <label for="">email</label>
    <input type="email" onfocus="this.style.backgroundColor='green'"
onblur="this.style.backgroundColor='blue'">
    <input type="submit">
  </form>
</body>
</html>
```

JS VARIABLES:

Definition

A **variable** in JavaScript is used to **store data values** so they can be reused and manipulated in a program.

Ways to Declare Variables

1. var

- Old way of declaring variables
- Can be updated
- Can be re-declared

var x = 10;

var x = 20; // allowed

x = 30; //allowed

2. let

- Can be updated
- Cannot be re-declared

```
let age = 21;
```

```
age = 22; // allowed
```

3. const

- Cannot be re-declared or reassigned
- Used for fixed values

```
const pi = 3.14;
```

Rules for Naming Variables

Must start with:

- A letter (a-z, A-Z)
- Underscore _
- Dollar sign \$

Cannot:

- Start with a number
- Use spaces
- Use JavaScript keywords

Valid variable names:

name

_age

\$total

userName

Invalid variable names:

1name

user name

let

JavaScript Data Types:

Definition

Data types define the **type of data** a variable can store.

JavaScript is **dynamically typed**, meaning you don't need to specify the data type.

```
let x = 10; // number
```

```
x = "Hello"; // string
```

Types of Data Types

1. Primitive Data Types

Type	Description	Example
String	Text	"Hello"
Number	Integer & decimal	10, 3.5
Boolean	True/False	true, false
Undefined	Value not assigned	let x;
Null	Empty value	null
BigInt	Large integers	123n
Symbol	Unique value	Symbol("id")

String

```
let name = "Reshma";
```

Number

```
let marks = 95;
```

```
let price = 99.99;
```

Boolean

```
let isPassed = true;
```

Undefined

```
let result;
```

Null

```
let data = null;
```

BigInt

```
let bigNum = 12345678901234567890n;
```

Symbol

```
let id = Symbol("id");
```

2. Non-Primitive (Reference) Data Types

Object

Stores multiple values as key-value pairs.

```
let student = {  
  name: "Ravi",  
  age: 21,  
  branch: "AI"  
};
```

Array

Stores multiple values in a list.

```
let subjects = ["HTML", "CSS", "JS"];
```

Function

A block of reusable code.

```
function greet() {  
  console.log("Hello");  
}
```

Checking Data Type

Use typeof operator:

```
typeof "Hello" // string  
typeof 10      // number  
typeof true   // boolean  
typeof null   // object (JS bug)
```

Operators:

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning	Example	Output
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (remainder)	10 % 3	1
**	Exponent	2 ** 3	8

let a = 10, b = 3;

console.log(a % b); // 1

2. Assignment Operators

Used to assign values.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2

3. Comparison Operators

Used to compare values (returns **true/false**).

Operator	Meaning	Example	Output
==	Equal (value only)	0 == "0"	true
===	Strict equal	5 === "5"	false
!=	Not equal	5 != 5	False
!==	Strict not equal	5 !== "5"	true

Operator	Meaning	Example	Output
>	Greater than	5 > 3	true
<	Less than	5 < 3	false

4. Logical Operators

Used with conditions.

Operator	Meaning	Example	Output
&&	AND	true && false	false
	OR	true false	true
!	NOT	!true	false

```
let age = 20;
```

```
console.log(age > 18 && age < 30); // true
```

5. Unary Operators

Work with **one operand**.

Operator	Example	Output
++	x++	Increase
--	x--	Decrease

```
let x = 5;
```

```
x++;
```

```
console.log(x); // 6
```

6. Ternary Operator

Shortcut for if-else.

```
let result = (age >= 18) ? "Adult" : "Minor";
```

7. Bitwise Operators (Advanced)

Used at **binary level**.

Operator	Name
&	AND

Operator	Name
	OR
^	XOR
<<	Left shift
>>	Right shift

8. String Operators

+ is also used for joining strings.

```
let name = "Mani";
```

```
let role = "Developer";
```

```
console.log (name + " is a " + role);
```

```
// Mani is a Developer
```

Type Conversion:

1. Implicit Type Conversion (Type Coercion)

Done automatically by JavaScript

JS converts types behind the scenes.

Examples

Expression	Output	Reason
5 + "5"	"55"	Number → String
"10" - 2	8	String → Number
"10" * "2"	20	String → Number
"10" / "2"	5	String → Number
true + 1	2	true → 1
false + 1	1	false → 0

```
console.log (5 + "5"); // "55"
```

```
console.log ("10" - 2); // 8
```

2. Explicit Type Conversion (Type Casting)

Done manually by the developer

String Conversion

Method	Example	Output
String()	String(100)	"100"
toString()	(100).toString()	"100"

let a = 10;

console.log(String(a)); // "10"

console.log((10).toString()); //"10"

Number Conversion

Method	Example	Output
Number()	Number("10")	10
parseInt()	parseInt("10.5")	10
parseFloat()	parseFloat("10.5")	10.5
Unary +	+"10"	10

console.log(Number("10")); // 10

console.log(parseInt("10.5")); // 10

console.log(+"20"); // 20

Invalid conversion → NaN (Not a Number)

Number("abc"); // NaN

Boolean Conversion

Value	Boolean Result
0	false
""	false
null	false
undefined	false
NaN	false
Others	true

Boolean(0); // false

Boolean("Hi"); // true

Truthy vs Falsy (Very Important)

Falsy Values (only 6)

false, 0, "", null, undefined, NaN

Truthy Values

"0", [], {}, "false", 1

Conditional Statements in JavaScript:

Conditional statements are used to **make decisions**.
They execute **different code blocks based on conditions**.

1. if Statement

Syntax:

```
if (condition) {  
  
    // code runs if condition is true  
  
}
```

Example

```
let age = 20;  
  
if (age >= 18) {  
  
    console.log("Eligible to vote");  
  
}
```

2. if - else Statement

Syntax:

```
if (condition) {  
  
    // true block  
  
} else {  
  
    // false block  
  
}
```

Example

```
let number = 5;

if (number % 2 === 0) {

  console.log("Even number");

} else {

  console.log("Odd number");

}
```

3. else if Statement

Used when **multiple conditions** are checked.

Syntax:

```
if (condition1) {

  // code

} else if (condition2) {

  // code

} else {

  // default

}
```

Example

```
let marks = 72;

if (marks >= 90) {

  console.log("Grade A");

} else if (marks >= 75) {

  console.log("Grade B");

} else if (marks >= 60) {

  console.log("Grade C");

} else {

  console.log("Fail");

}
```

```
}
```

4. switch Statement

Used when you want to compare **one value with many cases**.

Syntax:

```
switch (expression) {  
  case value1:  
    // code  
    break;  
  case value2:  
    // code  
    break;  
  default:  
    // code  
}
```

Example

```
let day = 3;  
switch (day) {  
  case 1:  
    console.log("Monday");  
    break;  
  case 2:  
    console.log("Tuesday");  
    break;  
  case 3:  
    console.log("Wednesday");  
    break;  
  default:  
    console.log("Invalid day");
```

```
}
```

Loops in JavaScript:

What is a Loop?

A loop is used to execute a block of code repeatedly until a condition becomes false.

Loops help to:

- Avoid writing the same code again and again
- Work with arrays and objects
- Perform repetitive tasks easily

Types of Loops in JavaScript:

1. for Loop

Definition:

Used when you know how many times the loop should run.

Syntax:

```
for(initialization; condition; increment/decrement) {  
  // code  
}
```

Example:

```
for(let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Output:

1 2 3 4 5

2. while Loop

Definition:

Runs the code while the condition is true. Used when the number of iterations is not known.

Syntax:

```
while(condition) {  
  // code  
}
```

Example:

```
let i = 1;
```

```
while(i <= 5) {  
  console.log(i);  
}
```

```
    i++;  
}
```

3. do...while Loop

Definition:

Executes the code at least once, then checks the condition.

Syntax:

```
do {  
    // code  
} while(condition);
```

Example:

```
let i = 1;  
  
do {  
    console.log(i);  
    i++;  
} while(i <= 5);
```

Loop Control Statements in JavaScript:

Loop control statements are used to **control the execution of loops**. They change the normal flow of a loop while it is running.

1. break Statement

Definition:

The break statement is used to **terminate (stop) the loop immediately**, even if the condition is still true.

Example:

```
for(let i = 1; i <= 5; i++) {  
    if(i == 3) {  
        break;  
    }  
    console.log(i);  
}
```

Output:

```
1  
2
```

2. continue Statement

Definition:

The continue statement is used to **skip the current iteration** of the loop and move to the next iteration.

Example:

```
for(let i = 1; i <= 5; i++) {  
    if(i == 3) {  
        continue;  
    }  
}
```

```
    console.log(i);  
}
```

Output:

```
1  
2  
4  
5
```

Function:

A **function** is a **block of reusable code** that performs a specific task. Instead of writing the same code again and again, we **call** a function.

Why Do We Use Functions?

- Avoid code repetition
 - Improve readability
 - Easy maintenance
 - Divide a big problem into small parts
-

Function Syntax

```
function functionName() {  
    // code to execute  
}  
functionName()
```

Example: Simple Function

```
function greet() {  
    console.log("Hello JavaScript");  
}
```

```
greet(); // function call
```

Function with Parameters

```
function greet(name) {  
    console.log("Hello " + name);  
}
```

```
greet("Ravi");  
greet("Sita");
```

name is a **parameter**
"Ravi" is an **argument**

Function with Return Value

```
function add(a, b) {  
  return a + b;  
}
```

```
let result = add(10, 20);  
console.log(result); // 30
```

return sends a value back to the caller
Code after return will not execute

Function with Default Parameter Value

```
function add(a, b=10) {  
  return a + b;  
}
```

```
let result = add(10);  
console.log(result); // 20
```

Types of Functions in JavaScript:

1. Named Function: function defined with a function name

```
function sayHello() {  
  console.log("Hello");  
}  
sayHello()
```

2. Function Expression: no name for a function (OR) Function assigned to variable

Anonymous Function

```
let sayHi = function () {  
  console.log("Hi");  
};
```

```
sayHi();
```

Arrow Function

```
const multiply = (a, b) => a * b;  
console.log(multiply(4, 5)); // 20
```

Object:

An **object** is a collection of **key - value pairs**.

Each key is called a **property**, and the value can be **any data type**.

Objects are used to represent **real-world entities**.

Real-World Example

A **student** has:

- name
- age
- marks
- isPassed

This fits perfectly as an object.

Creating an Object (Object Literal)

```
let student = {  
  name: "Ravi",  
  age: 20,  
  marks: 85,  
  isPassed: true  
};
```

Accessing Object Properties

1. Dot Notation (Most common)

```
console.log(student.name); // Ravi  
console.log(student.age); // 20
```

2. Bracket Notation

```
console.log(student["marks"]); // 85
```

Modifying Object Properties

```
student.age = 21;  
student.city = "Hyderabad";  
console.log(student);  
Objects are mutable (can be changed).
```

Deleting a Property

```
delete student.marks;
```

Object with Different Data Types

```
let user = {  
  name: "Sita",  
  age: 25,  
  skills: ["HTML", "CSS", "JS"],  
  address: {  
    city: "Chennai",  
    pin: 600001  
  }  
};
```

Accessing Nested Objects

```
console.log(user.address.city); // Chennai  
console.log(user.skills[1]);    // CSS
```

Check Object Type

```
typeof student; // "object"
```

Arrays and null also return "object".

Methods in object: function inside object are called methods

```
let user = {  
  name: "Ravi",  
  greet: function () {  
    console.log("Hello Ravi");  
  }  
};  
user.greet();
```

Using this in Object Methods

```
let user = {  
  name: "Sita",  
  greet: function () {  
    console.log("Hello " + this.name);  
  }  
};
```

user.greet(); // Hello Sita

this refers to the **current object**

Shorthand Method Syntax

```
let car = {  
  brand: "Tesla",  
  start() {  
    console.log(this.brand + " started");  
  }  
};  
car.start();
```

Object with Multiple Methods

```
let calculator = {  
  add(a, b) {  
    return a + b;  
  },  
  sub(a, b) {  
    return a - b;  
  },  
  mul(a, b) {  
    return a * b;  
  }  
};
```

```
console.log(calculator.add(10, 5)); // 15
```

Built-in Object Methods (Very Important)

Object.keys()

Returns all keys

```
let user = { name: "Ravi", age: 20 };  
console.log(Object.keys(user));  
// ["name", "age"]
```

Object.values()

Returns all values

```
console.log(Object.values(user));  
// ["Ravi", 20]
```

Object.entries()

Returns key–value pairs

```
console.log(Object.entries(user));  
// [["name", "Ravi"], ["age", 20]]
```

Looping Through Object

for in

```
for (let key in user) {  
    console.log(key, user[key]);  
}
```

User-Defined Objects

User-defined objects are **objects created by the programmer** to represent real-world entities with custom properties and methods.

JavaScript allows us to define our own object structure.

1. Properties

Properties are **variables inside an object**.
They store information about the object.

Example:

```
let student = {  
  
  name: "Ravi",  
  
  age: 20,  
  
  marks: 85  
  
};
```

Here:

- name, age, marks are **properties**
 - Property values can be of any data type
-

2. Methods

Methods are **functions inside an object**.
They define the behavior of an object.

Example:

```
let student = {  
  
  name: "Ravi",  
  
  greet: function () {  
  
    console.log("Hello Ravi");  
  
  }  
  
};
```

Calling a method:

```
student.greet();
```

Methods can use object properties using the **this** keyword.

3. Constructors

A **constructor** is a special function used to create **multiple objects with the same structure**.

Why constructor is needed?

Creating many objects manually is difficult:

```
let s1 = { name: "Ravi", age: 20 };
```

```
let s2 = { name: "Sita", age: 21 };
```

Instead, use a constructor.

Constructor Function Example

```
function Student(name, age, marks) {  
    this.name = name;  
    this.age = age;  
    this.marks = marks;  
}
```

Creating objects using constructor:

```
let s1 = new Student("Ravi", 20, 85);
```

```
let s2 = new Student("Sita", 21, 90);
```

```
console.log(s1.name); // Ravi
```

```
console.log(s2.marks); // 90
```

new keyword creates a new object
this refers to the current object being created

Adding Methods to Constructor

```
function Student(name, age) {  
    this.name = name;  
    this.age = age;  
    this.greet = function () {  
        console.log("Hello " + this.name);  
    };  
}
```

```
let s1 = new Student("Ravi", 20);
```

```
s1.greet(); // Hello Ravi
```

4. Accessors (get / set)

Accessors are **special methods** used to:

- get (read) property values
- set (update) property values

They provide **controlled access** to object data.

get Example

```
let user = {  
  firstName: "Ravi",  
  lastName: "Kumar",  
  get fullName() {  
    return this.firstName + " " + this.lastName;  
  }  
};  
  
console.log(user.fullName); // Ravi Kumar  
  
get is accessed like a property (not a function)
```

set Example

```
let user = {  
  name: "",  
  set userName(value) {  
    this.name = value;  
  }  
};  
  
user.userName = "Sita";
```



```
console.log(user.name); // Sita
```

get and set Together

```
let person = {  
  firstName: "Ravi",  
  lastName: "Kumar",  
  get fullName() {  
    return this.firstName + " " + this.lastName;  
  },  
  set fullName(name) {  
    let parts = name.split(" ");  
    this.firstName = parts[0];  
    this.lastName = parts[1];  
  }  
};  
  
person.fullName = "Sita Sharma";  
  
console.log(person.fullName); // Sita Sharma
```

Predefined (Built-in) Objects:

JavaScript provides **predefined (built-in) objects** to perform common tasks easily. These objects are already available in JavaScript and do not need to be created manually.

Array Object :

An **array** is a special variable that stores **multiple values** in a **single variable**. Each value has an **index** starting from 0

Example

```
let fruits = ["Apple", "Banana", "Mango"];
```

Index Value

0	Apple
1	Banana

Index	Value
2	Mango

Creating Arrays

1. Array Literal (Recommended)

```
let numbers = [10, 20, 30];
```

2. Using new Array ()

```
let colors = new Array("Red", "Green");
```

Accessing Array Elements

```
console.log(fruits[0]); // Apple
```

```
console.log(fruits[2]); // Mango
```

Modifying Array Elements

```
fruits[1] = "Orange";
```

```
console.log(fruits);
```

Array Length

```
console.log(fruits.length); // 3
```

Looping Through Arrays

for loop

```
for (let i = 0; i < fruits.length; i++) {  
  console.log(fruits[i]);  
}
```

for...of

```
for (let item of fruits) {  
  console.log(item);  
}
```

```
}
```

Common Array Methods: Arrays in JavaScript are objects and come with built-in methods.

Add Elements

```
fruits.push("Grapes"); // add at end
```

```
fruits.unshift("Kiwi"); // add at start
```

Remove Elements

```
fruits.pop(); // remove from end
```

```
fruits.shift(); // remove from start
```

Array with Mixed Data Types

```
let data = [10, "Hello", true, null];
```

JavaScript allows mixed types in arrays.

So, arrays in JavaScript are dynamic

Other Useful Methods

1. indexOf() – Find Index

```
let fruits = ["Apple", "Banana", "Mango"];
```

```
console.log(fruits.indexOf("Banana")); // 1
```

```
console.log(fruits.indexOf("Grapes")); // -1
```

2. includes() – Check Value

```
console.log(fruits.includes("Mango")); // true
```

3. slice() – Extract Part (Non-Destructive)

```
let nums = [10, 20, 30, 40];
```

```
let part = nums.slice(1, 3);
```

```
console.log(part); // [20, 30]
```

```
console.log(nums); // [10, 20, 30, 40]
```

4. splice() – Add / Remove (Destructive)

```
let nums = [10, 20, 30, 40];
```

```
nums.splice(1, 2);
```

```
console.log(nums); // [10, 40]
```

Removes 2 elements starting from index 1

10. sort() – Sort Elements

```
let names = ["Ravi", "Anil", "Sita"];
```

```
names.sort();
```

```
console.log(names); // ["Anil", "Ravi", "Sita"]
```

Numbers not possible

```
let nums = [10, 2, 5];
```

```
nums.sort();
```

```
console.log(nums); // [10, 2, 5]
```

String Object:

A string is a data type used to store text or characters.
Strings are immutable (cannot be changed directly).

Creating Strings

1. Using Single Quotes ' '

```
let name = 'Reshma';
```

2. Using Double Quotes " "

```
let city = "Hyderabad";
```

3. Using Template Literals `` (Recommended)

```
let course = `JavaScript`;
```

String Index

Each character in a string has an index starting from 0.

Example:

```
let word = "Hello";
```

Index Value

0 H

1 e

2 l

3 l

4 o

Accessing String Characters

```
console.log(word[0]); // H
```

```
console.log(word[4]); // o
```

Traditional String Concatenation: Using + operator

```
let firstName = "Rekha";
```

```
let lastName = "Dasari";
```

```
console.log(firstName + " " + lastName);
```

Using Template Literal (Modern Way)

```
let firstName = "Pallavi";
```

```
let age = 25;
```

```
console.log(`My name is ${firstName} and I am ${age} years old`);
```

String Length

```
let text = "JavaScript";
```

```
console.log(text.length); // 10
```

Looping Through Strings**for loop**

```
let str = "JS";
```

```
for (let i = 0; i < str.length; i++) {
```

```
  console.log(str[i]); }
```

for...of

```
for (let ch of str) {  
  console.log(ch);  
}
```

Common String Methods: Strings are primitive data types but JavaScript automatically treats them as objects when methods are used.

1. split() – Convert String to Array

```
let str = "HTML CSS JS";  
console.log(str.split(" "));  
// ["HTML", "CSS", "JS"]
```

2. slice() – Extract Part

```
let str = "JavaScript";  
console.log(str.slice(0, 4)); // Java
```

3. replace() – Replace Text

```
let str = "I love Java";  
console.log(str.replace("Java", "JS"));  
// I love JS
```

4. includes() – Check Value

```
let str = "Welcome to JavaScript";  
console.log(str.includes("JavaScript")); // true
```

5. startsWith()

```
let str = "JavaScript";  
console.log(str.startsWith("Java")); // true
```

6. endsWith()

```
let str = "JavaScript";  
console.log(str.endsWith("Script")); // true
```

7. toUpperCase()

```
let str = "hello";  
console.log(str.toUpperCase()); // HELLO
```

8. toLowerCase()

```
let str = "HELLO";  
console.log(str.toLowerCase()); // hello
```

9. trim() – Remove Extra Spaces

```
let str = " Hello JS ";  
console.log(str.trim()); // "Hello JS"
```

Multi-Line Strings:

1. Using Template Literal (Recommended)

```
let msg = `  
Hello Student  
Welcome to JavaScript  
Happy Learning  
`;  
console.log(msg);
```

2. Using New Line Character \n

```
let msg = "Hello Student\nWelcome to JavaScript\nHappy Learning";  
console.log(msg);
```

window Object:

The **window object** is the top-level object in the browser.
All global variables, functions, and objects are part of the window object.

Common window methods:

```
alert("Welcome to JavaScript");  
prompt("Enter your name");  
confirm("Are you sure?");
```

Other useful properties:

```
console.log(window.innerWidth);
```

```
console.log(window.innerHeight);
```

window is optional to write because it is the default object.

document Object

The **document object** represents the HTML page.
It is used to access and manipulate HTML elements.

Common document methods:

```
document.write("Hello World");
```

```
document.writeln("JavaScript Started");
```

Accessing page information:

```
console.log(document.title);
```

```
console.log(document.URL);
```

The document object helps JavaScript communicate with HTML.

Math Object

The **Math object** is used to perform mathematical operations.
It does not require object creation.

Common Math methods:

1. **Math.round()** – Rounds to nearest integer

```
Math.round(4.6); // 5
```

2. **Math.floor()** – Rounds down

```
Math.floor(4.9); // 4
```

3. **Math.ceil()** – Rounds up

```
Math.ceil(4.1); // 5
```

4. **Math.random()** – Generates random number (0–1)

```
Math.random();
```

5. **Math.max()** / **Math.min()**

```
Math.max(10, 20, 5); // 20
```

```
Math.min(10, 20, 5); // 5
```

Date Object

The **Date object** is used to work with dates and time.

Creating Date Object:

```
let today = new Date();
```

Common Date methods:


```
today.getDate();    // Day
today.getMonth();   // Month (0–11)
today.getFullYear(); // Year
today.getHours();   // Hours
today.getMinutes(); // Minutes
```

Example:

```
console.log(today.toString());
```

RegExp (Regular Expression) Object

RegExp (Regular Expression) is an object used to define a pattern for searching text.

It helps to:

- Validate forms (email, mobile, password)
 - Search text
 - Replace text
 - Extract data from strings
-

How to Create a Regular Expression

1. Using Literal (Recommended)

```
let pattern = /hello/;
```

2. Using Constructor

```
let pattern = new RegExp("hello");
```

Important RegExp Methods

1. test()

- Returns **true or false**
- Checks whether pattern exists

```
let text = "I love coding";
let pattern = /love/;
console.log(pattern.test(text));
// true
```

2. exec()

- Returns matched result as array
- Returns null if no match

```
let text = "Hello World";  
let pattern = /World/;  
console.log(pattern.exec(text));  
// ["World"]
```

String Methods That Use RegExp

1. match()

Returns matched values

```
let text = "apple banana apple";  
console.log(text.match(/apple/g));  
// ["apple", "apple"]
```

2. search()

Returns index of first match

```
let text = "Hello World";  
console.log(text.search(/World/));  
// 6
```

3. replace()

Replaces matched value

```
let text = "I like Java";  
console.log(text.replace(/Java/, "JavaScript"));  
// I like JavaScript
```

With global flag:

```
let text = "cat dog cat";  
console.log(text.replace(/cat/g, "tiger"));  
// tiger dog tiger
```

Flags in RegExp

Flag	Full Form	Meaning
i	Ignore case	Case-insensitive matching
g	Global (all matches)	Match all occurrences
m	Multiline	Multi-line matching

Example1:

```
let text = "hello Hello HELLO";
console.log(text.match(/hello/gi));
// ["hello", "Hello", "HELLO"]
```

Example 2: With i

```
let text = "JavaScript is Fun";
console.log(text.match(/javascript/i));
//["JavaScript"]
```

Example 3: With m

```
let text = `HTML
CSS
JavaScript`;
console.log(text.match(/^CSS/m));
//["CSS"]
```

Special Characters (Meta Characters)

These are powerful symbols:

Pattern Meaning

.	Any character
\d	Digit (0-9)
\D	Not digit
\w	Word character
\W	Not word character
\s	Space
\S	Not space

Pattern Meaning

^ Start of string

\$ End of string

Quantifiers

Used to define quantity

Symbol Meaning

+ One or more

* Zero or more

? Zero or one

{n} Exactly n times

{n,m} Between n and m

Example:

```
let text = "abc12345xyz";
console.log(text.match(/\d+/));
// ["12345"]
```

Form Validation Examples (Very Important)

1. Mobile Number (10 digits)

```
let mobile = "9876543210";
let pattern = /^\d{10}$/;
console.log(pattern.test(mobile));
// true
```

Explanation:

- ^ → start
 - \d{10} → exactly 10 digits
 - \$ → end
-

2. Email Validation

```
let email = "reshma@gmail.com";
let pattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}$/;
```

```
console.log(pattern.test(email));  
// true
```

Character Sets

Using square brackets []

```
let text = "bat cat mat";  
console.log(text.match(/[bcm]at/g));  
// ["bat", "cat", "mat"]
```

Matches any word starting with b, c, or m.

Grouping (Important Concept)

Using parentheses ()

```
let text = "2026-02-11";  
let pattern = /(\d{4})-(\d{2})-(\d{2})/;  
let result = text.match(pattern);  
console.log(result[0]); // 2026-02-11  
console.log(result[1]); // year  
console.log(result[2]); // month  
console.log(result[3]); // day
```

Real-Life Example (Form Validation)

```
function validateName(name) {  
  let pattern = /^[A-Za-z ]{3,}$/;  
  return pattern.test(name);  
}  
console.log(validateName("Reshma"));  
// true
```

DOM Basics

DOM stands for **Document Object Model**.

It is a programming interface that allows JavaScript to **access, change, and manipulate HTML elements**.

HTML is converted into objects so JavaScript can work with it.

What is DOM

When a web page loads, the browser creates a **DOM** of the page.

In simple words:

HTML → DOM (Object Model) → JavaScript

Using DOM, JavaScript can:

- Change HTML content
- Change styles
- Add or remove elements
- Handle user actions

Example:

```
document.title = "DOM Started";
```

Here, JavaScript changes the web page title using DOM.

DOM Tree

The DOM represents an HTML document in a **tree structure**.

- Every HTML element is a **node**
- Elements are connected in a **parent–child relationship**

Example HTML:

```
<html>
  <body>
    <h1>Hello</h1>
    <p>Welcome</p>
  </body>
</html>
```

DOM Tree Representation:

- html (root)
 - body (parent)
 - h1 (child)
 - p (child)

body is the parent

h1 and p are children

h1 and p are siblings

DOM Manipulation

DOM manipulation means **changing HTML elements using JavaScript**.

Changing HTML Content

```
document.getElementById("demo").innerHTML = "Hello DOM";
```

Changing Styles

```
document.getElementById("demo").style.color = "red";  
document.getElementById("demo").style.fontSize = "20px";
```

Changing Attributes

```
document.getElementById("img").src = "image.jpg";
```

Creating and Adding Elements

```
let para = document.createElement("p");  
para.innerHTML = "New paragraph";  
document.body.appendChild(para);
```

Removing Elements

```
let element = document.getElementById("demo");  
element.remove();
```

Main Points:

- DOM connects **HTML and JavaScript**
 - DOM is created automatically by the browser
 - HTML elements become **objects**
 - DOM tree shows parent–child relationships
 - DOM manipulation allows dynamic web pages
-

DOM Selection

DOM selection is used to **select HTML elements** so that JavaScript can **read, modify, or manipulate** them.

To work with any HTML element using JavaScript, **we must select it first**.

getElementById()

Selects **one element** using its id.

- id must be **unique**

- Returns a **single element**

Example HTML:

```
<h1 id="title">Welcome</h1>
```

JavaScript:

```
let heading = document.getElementById("title");
```

```
heading.innerHTML = "DOM Selection Started";
```

Most commonly used and easiest method.

getElementsByName()

Selects **multiple elements** using a class name.

- Returns an **HTMLCollection**
- Access elements using index

Example HTML:

```
<p class="text">HTML</p>
```

```
<p class="text">CSS</p>
```

```
<p class="text">JS</p>
```

JavaScript:

```
let items = document.getElementsByName("text");
```

```
items[0].style.color = "red";
```

```
items[1].style.color = "blue";
```

```
items[2].style.color = "green";
```

HTMLCollection is **array-like**, not a real array.

getElementsByTagName()

Selects **multiple elements** using tag name.

- Returns an **HTMLCollection**

Example HTML:

```
p
```

JavaScript:

```
let paras = document.getElementsByTagName("p");
```

```
paras[0].innerHTML = "First paragraph";
```

querySelector()

Selects **the first matching element** using a **CSS selector**.

- Uses # for id
- Uses . for class
- Can select tags also

Examples:

```
document.querySelector("#title"); // id
```

```
document.querySelector(".text"); // class
```

```
document.querySelector("p"); // tag
```

Example:

```
let el = document.querySelector(".text");
```

```
el.style.fontWeight = "bold";
```

Modern and very powerful method.

querySelectorAll()

Selects **all matching elements** using a **CSS selector**.

- Returns a **NodeList**
- Can be looped using for or forEach

Example:

```
let items = document.querySelectorAll(".text");
```

```
items.forEach(function (item) {  
  item.style.color = "purple";  
});
```

DOM Events

DOM events occur when a **user interacts with a web page**.
JavaScript responds to these actions using **event handling**.

Examples of user actions:

- Clicking a button
- Typing in an input box
- Submitting a form
- Moving the mouse

Event Listeners

An **event listener** waits for an event to happen and then executes a function.

Modern way of handling events:

```
element.addEventListener("event", function);
```

Example:

```
<button id="btn">Click Me</button>

let btn = document.getElementById("btn");
btn.addEventListener("click", function () {
  console.log("Button clicked");
});
```

Multiple event listeners can be added to the same element
Preferred over inline events

Click Events

Click events occur when an element is clicked.

Common click event:

- click

Example:

```
<button id="clickBtn">Click</button>

let button = document.getElementById("clickBtn");

button.addEventListener("click", function () {
  button.innerHTML = "Clicked!";
});
```

Changing styles on click:

```
button.addEventListener("click", function () {
  button.style.backgroundColor = "green";
  button.style.color = "white";
});
```

Form Events

Form events occur when a user interacts with a form.

Common form events:

- submit
- change

- focus
 - blur
 - input
-

submit Event

Triggered when a form is submitted.

Example:

```
<form id="myForm">
  <input type="text" id="name">
  <button type="submit">Submit</button>
</form>

let form = document.getElementById("myForm");
form.addEventListener("submit", function (event) {
  event.preventDefault();
  console.log("Form submitted");
});
```

➔ event.preventDefault() stops page reload

change Event

Triggered when the value of an input changes.

```
<input type="text" id="username">

let input = document.getElementById("username");
input.addEventListener("change", function () {
  console.log("Input value changed");
});
```

focus and blur Events

- focus → when input is selected
- blur → when input loses focus

```
input.addEventListener("focus", function () {
  input.style.backgroundColor = "lightyellow";
});

input.addEventListener("blur", function () {
```

```
input.style.backgroundColor = "white";  
});
```

Main Points:

- Events make web pages **interactive**
- `addEventListener()` is the modern approach
- Click events handle button interactions
- Form events handle user input
- `preventDefault()` stops default browser behavior